

Smart Recycling and Food Waste Prevention System

Final Project Documentation

Or Gabbay (211906508) & Daniel Yerichman (318851623)

2025 -2026

Part 1: Project Overview

What Is This Project and Why Did We Choose It?

We all know the moment standing in front of the recycling bins: holding a package and not sure whether it goes in the orange bin, the blue bin, or the regular trash. This confusion causes many recyclable materials to simply go to waste.

But when we thought more deeply about the waste problem, we realized there is an even bigger issue: food waste. We often throw away fruits and vegetables just because they look a bit tired, or conversely, we fail to notice that a piece of fruit is rotting and contaminating the entire fruit basket.

Our idea is to develop a technological tool that helps people manage their kitchen and their waste more intelligently.

What Does the System Actually Do?

We are developing software based on Computer Vision that can "look" at objects and give the user an immediate answer in two modes:

1. **Waste Sorting:** The system identifies the type of material (glass, plastic, paper) and clearly indicates which colored bin it should go into.
2. **Waste Prevention (Our Innovation):** The system identifies fruits and vegetables and can distinguish between fresh produce, produce that is about to spoil, and rotten produce. This helps the user decide what to eat today, what to cook, and what must go to compost.

Our Challenge: Building from Scratch

To make the project original and challenging, we decided not to use pre-built models from major companies. Instead, we build and train the Deep Learning models ourselves, from scratch.

We want to tackle the challenge of teaching a computer to distinguish between subtleties — for example, the small difference in color and texture between a ripe banana and a rotten banana.

The Bottom Line

The goal is to create a web application that not only helps with recycling, but helps reduce the amount of garbage we produce in the first place. This is a combination of advanced technology, financial savings, and environmental protection. The application runs in the browser on any device — no app store download required.

Part 2: Introduction

Project Overview

The accumulation of waste and the inefficiency of recycling processes are significant global challenges. In daily life, consumers frequently face confusion regarding proper waste sorting — uncertain whether an item belongs in the orange, blue, or general waste bin. This confusion leads to contamination of recycling streams and the loss of recyclable materials.

However, a deeper analysis of household waste reveals an even more critical issue: food waste. Households often discard fruits and vegetables prematurely due to aesthetic imperfections or, conversely, fail to identify spoiling produce in time, causing cross-contamination of fresh food.

Our project aims to address these dual challenges by developing a smart web application based on Computer Vision and Deep Learning. The system will provide users with immediate, visual-based feedback in two scenarios:

- **Waste Sorting:** Identifying the material (plastic, glass, paper) and directing the user to the correct recycling bin.
- **Food Waste Prevention:** Classifying produce into distinct freshness categories (e.g., Fresh, Half-Fresh, Rotten) to guide consumption decisions (e.g., "eat now," "cook immediately," or "compost").

The Challenge and Innovation

The primary challenge of this project is the decision to construct and train the Deep Learning models from scratch, rather than relying on pre-built commercial APIs (such as Google Lens or AWS Rekognition). This requires implementing complex Convolutional Neural Networks (CNNs) capable of distinguishing subtle visual nuances, such as the texture difference between a ripe banana and a spoiling one.

The innovation lies in the integration of waste classification with proactive freshness detection. While many solutions focus solely on the "end-of-life" (disposal) of a product, our system aims to intervene earlier in the cycle, extending the lifespan of food products and preventing waste generation at the source.

Part 3: Competitor Analysis and Related Work

Current Market Landscape

The domain of waste management technology is currently divided into industrial solutions and consumer-facing applications.

- **Industrial Solutions:** Large-scale waste sorting facilities utilize advanced robotics and sensors (NIR, hyperspectral imaging) to sort waste on conveyor belts. These systems are highly accurate but are not applicable to the domestic user.
- **Consumer Apps:** Existing applications for recycling mostly rely on barcode scanning or manual search databases. These methods are cumbersome and often fail when packaging is damaged or barcodes are missing.

The Technological Gap

Regarding Computer Vision solutions, research exists in isolated domains. For instance, there are academic implementations of "Smart Bins" that classify trash. However, these are rarely integrated with food freshness analysis. On the food waste front, most "smart kitchen" apps function as inventory trackers where users must manually input expiry dates. There is a lack of accessible, real-time visual analysis tools that can objectively determine the freshness of specific produce items for the home user.

Our Contribution

Our project bridges this gap by combining Recycling Guidance and Freshness Detection into a single, cohesive web interface. Unlike industrial systems that prioritize throughput, our model prioritizes the distinction between "edible" and "waste," dealing with the ambiguity of organic matter. By building the model from scratch, we aim to optimize the architecture specifically for these distinct classes, avoiding the computational overhead of generic, large-scale models.

Part 4: Literature Review

This section reviews the theoretical foundations and state-of-the-art methodologies relevant to our project. The review is categorized into three major themes: Deep Learning for Waste Classification, Algorithms for Food Freshness Detection, and Technical Architecture for Custom Model Construction.

A. Deep Learning Architectures for Waste Classification

To achieve accurate waste sorting, it is essential to understand how Convolutional Neural Networks (CNNs) handle diverse object types. Stanford University's CS231N project demonstrates the efficacy of CNNs in categorizing waste into distinct material classes (e.g., metal, paper, plastic) even under challenging conditions [3]. This work highlights the importance of specific layer configurations in extracting features from images of crumpled or damaged trash, which is a common scenario in our use case.

Furthermore, Qiao's comparative analysis [4] provides a critical evaluation of different architectures, including AlexNet, ResNet, and Vision Transformers. The study suggests that while Transformers offer high accuracy, optimized CNNs (like ResNet) often provide a better balance of performance and computational efficiency for specific tasks. This supports our potential choice of a residual network structure for the recycling component.

In terms of achieving high accuracy, Ahmad et al. [5] presented an intelligent waste sorting system published in Nature Scientific Reports, which utilized an optimized ResNet-based architecture. Their model achieved over 98% accuracy, significantly outperforming previous conventional architectures. Their work emphasizes the need for careful hyperparameter tuning and data augmentation to handle the visual variety of trash items.

Finally, for the application to be user-friendly, real-time detection is crucial. The work by detecting waste using the YOLO (You Only Look Once) framework [9] illustrates how object detection models can identify multiple waste items in a single frame with low latency. This is particularly relevant for our web app, which aims to provide immediate feedback to the user.

B. Methodologies for Food Freshness Detection

The "Food Waste Prevention" component of our project requires distinguishing between subtle states of organic decay. The UNEP Food Waste Index Report 2024 [2] provides the critical context and statistical framework for this domain. The report highlights that households are responsible for a significant portion of global food waste, validating the urgency of consumer-facing technological interventions.

More specifically regarding the algorithms, the study "Detecting Freshness of Food Using AI" [6] is directly aligned with our project goals. It proposes a classification system with granular labels: "Fresh," "Half-Fresh," and "Spoiled." This granularity is vital for our user interface, as "Half-Fresh" items should be recommended for cooking rather than disposal.

Similarly, Zhang et al. [7] explored Multi-Task Convolutional Neural Networks, which simultaneously identify the fruit type and its freshness level. Adopting a multi-task learning

approach could allow our system to be more efficient, running one pass of the network to answer both "What is this?" and "Is it good to eat?"

The necessity of building a custom model is further supported by Aksoy et al. [8], who evaluated pretrained CNNs for distinguishing fresh versus rotten produce. Their findings indicate that while pretrained models (transfer learning) are powerful, specific datasets regarding rot and decay require significant fine-tuning. This validates our decision to train our models from scratch to capture the specific visual features of decay that general models might miss.

C. Technical Implementation and Model Construction

As our challenge involves building models from the ground up, foundational resources are critical. The recent textbook "Understanding Deep Learning" by Prince [1] serves as our primary technical guide. Published in 2023, it offers up-to-date methodologies for implementing Deep Learning architectures. It specifically details the mathematical operations behind CNNs and optimization algorithms, which are essential for our goal of training a model from scratch rather than using pre-built APIs.

Additionally, the interactive resource "Dive into Deep Learning" [10] provides the necessary mathematical and code-based foundations for implementing the backpropagation and optimization algorithms required for training. These resources are essential for ensuring that our custom-built models are mathematically sound and optimized for performance.

Conclusion of Review

The literature confirms that while waste classification and freshness detection are researched individually, there is a clear opportunity to combine them into a single consumer-centric application. The research supports our hypothesis that a custom-trained CNN, optimized for these specific tasks, can achieve the necessary accuracy to assist users in reducing their environmental footprint.

Part 5: References

- [1] S. J. D. Prince, Understanding Deep Learning. MIT Press, 2023. Available: <https://udlbook.github.io/udlbook/>
- [2] UNEP, Food Waste Index Report 2024. Nairobi: United Nations Environment Programme, 2024. Available: <https://www.unep.org/resources/publication/food-waste-index-report-2024>
- [3] Stanford CS231N Team, "Waste Classification and Management Using Computer Vision," in Stanford CS231N Final Project Papers, 2025.
- [4] Z. Qiao, "Advancing Recycling Efficiency: A Comparative Analysis of Deep Learning Models in Waste Classification," arXiv preprint arXiv:2411.02779, 2024.
- [5] G. Ahmad et al., "Intelligent waste sorting for urban sustainability using deep learning," Sci. Rep., vol. 15, no. 1, Art. no. 27078, Jul. 2025.
- [6] "Detecting Freshness of Food Using AI – CNN and Random Forest Integration," International Journal of Research Publication and Reviews, vol. 6, no. 5, 2025.
- [7] Y. Zhang et al., "Fruit Freshness Detection Based on Multi-Task Convolutional Neural Networks," PMC, 2024.
- [8] S. Aksoy et al., "Evaluating Pretrained CNNs for Distinguishing Fresh vs Rotten Fruits and Vegetables," Horticulture Research, 2024.
- [9] "AI-Powered Autonomous Waste Detection System Using Deep Learning (YOLO Framework)," International Journal for Multidisciplinary Research, 2025.
- [10] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, Dive into Deep Learning, 2024. Available: <https://d2l.ai>

Part 6: Functional and Technological Requirements Specification

1. Functional Requirements

This section defines the features and capabilities the system must possess from the end-user's perspective, prioritizing interactions that reduce waste at the source.

1.1 Identify Stakeholders

- **End Users:** Home cooks and environmentally conscious individuals seeking to reduce household waste and improve recycling habits.
- **Project Team:** Data Science Students (Or & Daniel) responsible for the end-to-end development, specifically the training of custom CNN models.

1.2 User Stories & Prioritization

Story 1: Waste Identification

As a user holding a piece of trash, I want to point my camera at the item, so that I know exactly which colored bin (orange, blue, general) it belongs to.

Acceptance Criteria: The system correctly identifies plastic, glass, or paper and displays the corresponding bin color on the screen within 3 seconds.

Priority: High (Critical) | **Estimated Effort:** 5 Days

Story 2: Freshness Detection

As a home cook, I want to scan a fruit or vegetable, so that I can determine its freshness level (Fresh, Half-Fresh, or Rotten).

Acceptance Criteria: The app displays a label classifying the produce freshness and provides a confidence percentage.

Priority: High (Critical) | **Estimated Effort:** 7 Days

Story 3: Actionable Advice

As a user trying to reduce food waste, I want to receive specific advice based on the freshness of the item, so that I know whether to eat it now, cook it immediately, or compost it.

Acceptance Criteria: If "Half-Fresh" is detected, the system suggests "Cook Immediately"; if "Rotten" is detected, it suggests "Compost."

Priority: Medium | **Estimated Effort:** 2 Days

Story 4: Custom Model Feedback (Training Loop)

As a user, I want to correct the system if it misidentifies an object, so that the custom Deep Learning model can improve over time.

Acceptance Criteria: A "Report Incorrect Result" button is available after every scan.

Priority: Low (Future Enhancement) | **Estimated Effort:** 4 Days

1.3 Use Cases

Use Case 1: Recycling Assistance

- **Actor:** User with waste item.
- **Pre-conditions:** User has opened the web app; Camera permissions are granted in the browser.
- **Basic Flow:**
 1. User selects "Recycle Scan" mode.
 2. User points camera at the object (e.g., a plastic bottle).
 3. System captures the image and processes it using the custom CNN model.
 4. System identifies the material (Plastic).
 5. System displays "Plastic — Orange Bin" overlay on the screen.
- **Alternate Flow — Unknown Object:** System displays "Object not recognized. Please try again."

Use Case 2: Produce Freshness Check

- **Actor:** User with produce (e.g., Banana).
- **Pre-conditions:** User has opened the web app; Lighting is adequate.
- **Basic Flow:**
 1. User selects "Freshness Scan" mode.
 2. User aligns the fruit within the frame.
 3. System analyzes texture and color nuances.
 4. System returns status: "Half-Fresh."
 5. System prompts: "Best used for baking (e.g., Banana Bread)"
- **Alternate Flow — Blurred Image:** System requests the user to steady the camera.

1.4 Core Functions

- **User Authentication:** Secure login/registration to save history.
- **Inference Engine:** The core logic running custom CNNs to classify images without external commercial APIs.
- **Result Visualization:** Visual overlays indicating bin color or freshness status.
- **Scan History:** A log of scanned items to track personal waste reduction.

2. Architectural Requirements

2.1 System Architecture: Client-Server

Given the need for heavy Deep Learning processing (CNNs) and the requirement for cross-device accessibility, a Client-Server Architecture is selected.

- **Client (Web App):** The frontend interface built with React (Vite) + TailwindCSS. Handles the user interface, image capture via the browser's MediaDevices API, and results display. It ensures a responsive experience on any device (mobile, tablet, desktop) while offloading heavy computation to the server.
- **Server (Backend + AI):** Hosts the custom-trained models. It receives images, performs inference (prediction), and returns the results.

2.2 Sequence of Interactions

The following describes the system sequence for a typical scan operation:

1. User initiates a scan on the Web App.
2. Web App compresses the image and sends a POST request to the API Gateway.
3. API Gateway validates the user token and forwards the image to the Inference Service.
4. Inference Service pre-processes the image (using OpenCV) and feeds it into the trained CNN model.
5. Model returns the predicted class (e.g., "Rotten") and a confidence score (e.g., "95%").
6. Server logs the transaction and image path in the Database (for future retraining).
7. Server sends the JSON response back to the Web App.
8. Web App renders the result to the user.

2.3 Performance Requirements

- **Latency:** The system must provide visual feedback in under 2 seconds to ensure the "immediate" feeling described in the project goals.
- **Accuracy:** The Freshness Model should distinguish between "Fresh" and "Rotten" with at least 85% accuracy, addressing the difficulty of detecting subtle decay.
- **Concurrency:** The backend should support at least 100 concurrent requests without crashing, scalable via cloud services.

2.4 Reliability and Availability

- **Availability:** The system targets 99% uptime.
- **Failover:** In case of backend failure, the web app will catch the exception and display a user-friendly "Service Temporarily Unavailable" message within 5 seconds.
- **Data Integrity:** User data and collected images for the retraining dataset will be backed up daily to cloud storage (AWS S3).

2.5 Data Architecture

- **User Data:** SQL Database (PostgreSQL) for profiles and history.
- **Image Repository:** Object Storage (S3) is crucial. Since models are built "from scratch," every user scan is anonymously saved to a "Retraining Dataset" to improve the model's ability to distinguish nuances like texture.
- **Labeling Data:** Stores the classification results and any user corrections (Ground Truth).

3. Technological Requirements

3.1 Frontend Technologies (Web App)

Framework: React (Vite) + TailwindCSS

Reasoning: A responsive web application ensures the system is accessible from any device with a modern browser — smartphones, tablets, and desktops — without the need to publish through app stores (Apple App Store, Google Play). Vite provides fast build times and an excellent developer experience. TailwindCSS enables rapid, consistent UI development.

Camera Access: Browser MediaDevices API (`navigator.mediaDevices.getUserMedia`) to access the device camera directly from the web app for real-time image capture.

3.2 Backend Technologies

Language: Python

Reasoning: The primary language for Deep Learning and Computer Vision research and implementation.

Web Framework: FastAPI

Reasoning: High performance, asynchronous support, and excellent for serving Machine Learning models via REST APIs.

3.3 AI & Machine Learning Stack (The "From Scratch" Component)

Deep Learning Framework: PyTorch

Reasoning: Essential for building and training the Convolutional Neural Networks (CNNs) and implementing backpropagation algorithms as detailed in "Understanding Deep Learning" by Prince.

Computer Vision Library: OpenCV

Reasoning: Required for image pre-processing (resizing, normalization, noise reduction) before feeding images into the CNN.

Model Architecture: ResNet or MobileNet

Reasoning: Literature suggests optimized CNNs like ResNet provide a better balance of performance and efficiency for waste classification compared to generic Transformers.

3.4 Database & Storage

Primary DB: PostgreSQL — reliable relational database for user management and scan metadata.

Object Storage: AWS S3 — to store the actual image files generated by users for the dataset.

3.5 Cloud & Infrastructure

Platform: AWS (Amazon Web Services) — allows for scalable hosting and access to GPU instances (e.g., EC2 g4dn) required for training or fast inference.

Containerization: Docker — ensures the Python environment and dependencies are consistent between the development machine and the production server.

3.6 Testing and Quality Assurance

- **Unit Testing:** PyTest for backend logic validation.
- **Model Evaluation:** The AI models will be evaluated using a held-out Test Set to calculate the Confusion Matrix, Precision, and Recall, ensuring the model generalizes well to new images.
- **User Acceptance Testing (UAT):** Manual testing in various lighting conditions to verify the app's robustness in a real kitchen environment.

3.7 Security & Compliance

- **Data Encryption:** HTTPS for data in transit; Encryption at rest for user databases.
- **Authentication:** Secure authentication (OAuth2 or JWT) to protect user accounts and history.

Part 7: Datasets

Garbage Classification V2 (Kaggle)

This dataset is designed for image classification tasks related to waste management and recycling. It contains 19,762 images distributed across 10 distinct classes: Metal, Glass, Biological, Paper, Battery, Trash, Cardboard, Shoes, Clothes, and Plastic. The images are well-suited for training deep learning models to automatically categorize different types of household garbage.

URL: <https://www.kaggle.com/datasets/sumn2u/garbage-classification-v2>

TACO Dataset

TACO is an open-source image dataset focusing on litter detection "in the wild" (e.g., on streets, beaches, and in forests), rather than clean, isolated items. It features high-resolution images with pixel-level annotations (segmentation masks) for training object detection and segmentation algorithms. The dataset covers 60 categories of litter and uses a hierarchical taxonomy.

URL: <http://tacodataset.org/>

Mendeley FruitVision

Hosted on Mendeley Data, this dataset is designed to help quality control systems distinguish between fresh and spoiled produce. The dataset includes images of 5 types of fruits: Apples, Bananas, Mangoes, Oranges, and Grapes. It categorizes them into classes such as Fresh, Rotten, and (in some versions) Formalin-mixed, providing a robust benchmark for evaluating freshness.

URL: <https://data.mendeley.com/datasets/xkbjx8959c/2>

Part 8: Detailed Design

1. Executive Summary

This project outlines the design of a web application aimed at solving two critical environmental problems: confusion regarding waste sorting (recycling) and household food waste. Unlike standard solutions that rely on commercial APIs, this system utilizes custom-trained Convolutional Neural Networks (CNNs) to analyze images in real-time. The system identifies waste materials to direct users to the correct bin (Orange, Blue, General) and classifies produce freshness (Fresh, Half-Fresh, Rotten) to suggest actionable consumption advice. The application is delivered as a responsive web app accessible from any modern browser.

2. System Architecture

The system follows a Client-Server Architecture. This design was selected to offload the heavy computational load of Deep Learning processing from the user's device to a dedicated server, ensuring the web app remains lightweight and responsive.

2.1 High-Level Components

- **Client (Web App):** The frontend interface built with React (Vite) + TailwindCSS, responsible for image capture via the browser's MediaDevices API, user interaction, and displaying results.
- **API Gateway:** The entry point for the backend that handles authentication, validation, and request routing.
- **Inference Service (AI Engine):** The core processing unit hosting the custom CNN models (ResNet/MobileNet) to perform predictions.
- **Data Layer:** Consists of a PostgreSQL database for metadata and AWS S3 for image storage.

3. Module Design

3.1 Frontend Subsystem (Web App)

Technology: React (Vite) + TailwindCSS

Key Responsibilities:

- **Camera Interface:** Uses the browser's MediaDevices API (`navigator.mediaDevices.getUserMedia`) to capture images directly from the device camera.
- **Compression:** Compresses images using JavaScript Canvas API before transmission to minimize latency.
- **User Modes:** (1) Recycle Scan — distinct UI for waste identification. (2) Freshness Scan — distinct UI for produce analysis.

- **Result Visualization:** Renders overlays (e.g., "Plastic — Orange Bin" or "Half-Fresh — Cook Immediately").
- **Responsive Design:** Fully responsive layout that adapts to mobile, tablet, and desktop screen sizes.

3.2 Backend Subsystem (API & Logic)

Technology: Python with FastAPI.

Concurrency: Designed to support at least 100 concurrent requests via asynchronous processing.

Logic Flow:

1. Receive POST request with the compressed image.
2. Validate User Token (OAuth2/JWT).
3. Route to the specific model (Waste vs. Freshness) based on the user's selected mode.

3.3 AI Inference Engine (The "From Scratch" Core)

Framework: PyTorch

Pre-processing: Uses OpenCV to resize, normalize, and reduce noise in images before they enter the neural network.

Model Architecture: ResNet or MobileNet. Literature indicates ResNet provides an optimal balance of accuracy and efficiency for waste classification compared to heavier Transformers.

Output: Returns a JSON object containing the Class (e.g., "Rotten") and Confidence Score (e.g., "95%").

4. Data Design and Management

4.1 Datasets (Training Material)

Waste Classification:

Garbage Classification V2 (Kaggle): 19,762 images across 10 classes. TACO Dataset: Used for "wild" litter detection to improve real-world accuracy.

Freshness Detection:

Mendeley FruitVision: Images of apples, bananas, mangoes, etc., categorized into "Fresh" and "Rotten".

4.2 Database Schema (PostgreSQL)

Users Table: Stores credentials and profile data. Scan History Table: Logs every transaction including Timestamp, Item Detected, Confidence Score, and Action Taken.

4.3 Image Repository (AWS S3)

Purpose: The system implements a "Training Loop." Every user scan is anonymously saved to S3.

Benefit: These real-world images allow for continuous retraining and fine-tuning of the custom model, specifically to improve texture recognition (e.g., distinguishing spotting on a banana).

5. Process Logic (Sequence of Operations)

The following sequence describes the "Happy Path" for a freshness scan:

9. Initiation: User selects "Freshness Scan" and points the camera at a fruit.
10. Capture & Send: The web app captures the image, compresses it via Canvas API, and sends a POST request to the API Gateway.
11. Processing: The Inference Service receives the image. OpenCV performs pre-processing. The Custom CNN predicts the class "Half-Fresh" with 85%+ confidence.
12. Logging: The server logs the transaction in PostgreSQL and saves the image path to S3.
13. Response: The server returns the result to the web app.
14. Action: The web app displays "Half-Fresh" and suggests: "Best used for baking (e.g., Banana Bread)".

6. Development & Testing Plan

- **Environment:** Docker containers will be used to ensure consistency between development and the production server (AWS EC2).
- **Model Evaluation:** The AI models will be evaluated using a held-out Test Set. Metrics include Confusion Matrix, Precision, and Recall to ensure the model generalizes well.
- **Performance Goals:** Latency: Visual feedback must be rendered in under 2 seconds. Accuracy: Freshness detection must exceed 85% accuracy.